UNIVERSIDAD VILLASUNCIÓN
CLASE 7 · CUADERNO DE EJERCICIOS

Dependencia, asociación, UML, Python y control de versiones

Asignatura **Programación Orientada a Objetos**	**Clave** **LCF416**	**Clase** **7**	**Unidad** **III**
Temas **3.1 Dependencia /** **3.2 Asociación**	**Lenguaje** **Python**	**Modalidad** **Mixta**	**Duración** **2 horas**

Nombre del estudiante: Daniel Sánchez Sánchez
Materia: Programación Orientada a Objetos
Docente: Jorge Alberto Méndez de Ávila
Grupo: 4to. “A”
Fecha de entrega: 24/ 10 / 2026

DATOS DE ENTREGA

Cómo trabajar este cuaderno

Los ejercicios avanzan desde el razonamiento del modelo hasta una evidencia profesional en GitHub. Trabaja en orden. Los ejercicios 1 al 8 se consideran práctica obligatoria; los ejercicios 9 y 10 son de consolidación; el ejercicio 11 constituye el avance del proyecto integrador.

Regla de trabajo
No copies únicamente comandos. Antes de ejecutar cada paso explica qué esperas que ocurra y verifica el resultado con git status o con la interfaz de GitHub.

Ejercicio 1. Clasifica la relación

Para cada caso indica si representa principalmente DEPENDENCIA o ASOCIACIÓN y justifica en una oración:

A) ReportePDF recibe un Incidente para generar un archivo.

B) Incidente conserva un atributo responsable de tipo Usuario.

C) GestorIncidentes recibe un Notificador solo cuando cierra un caso.

D) Usuario mantiene una lista de incidentes asignados.

Respuesta / desarrollo:

A) ReportePDF recibe un Incidente para generar un archivo.

Respuesta: DEPENDENCIA, porque ReportePDF utiliza un objeto Incidente para generar el archivo, sin necesidad de conservarlo como atributo.

B) Incidente conserva un atributo responsable de tipo Usuario.

Respuesta: ASOCIACIÓN, porque la clase Incidente mantiene una referencia a un objeto Usuario mediante el atributo responsable.

C) GestorIncidentes recibe un Notificador solo cuando cierra un caso.

Respuesta: DEPENDENCIA, porque GestorIncidentes utiliza el objeto Notificador temporalmente para enviar la notificación al cerrar un incidente.

D) Usuario mantiene una lista de incidentes asignados.

Respuesta: ASOCIACIÓN, porque la clase Usuario mantiene una relación con varios objetos Incidente mediante una lista de incidentes asignados.

Ejercicio 2. Del lenguaje natural al modelo

Lee: "Un incidente puede estar sin responsable o tener un responsable. Un usuario puede ser responsable de muchos incidentes". Escribe las multiplicidades en ambos extremos y explica por qué.

Respuesta / desarrollo:

La multiplicidad del extremo Incidente es 0..*, porque un usuario puede ser responsable de cero, uno o muchos incidentes. La multiplicidad del extremo Usuario es 0..1, porque un incidente puede no tener responsable o tener solamente uno.

Ejercicio 3. Diseña la clase Usuario

Define atributos y métodos mínimos de Usuario. Incluye tipos de datos y visibilidad UML. Debe poder presentar un resumen del usuario.

Respuesta / desarrollo:

```python
class Usuario:
    def __init__(self, id: int, nombre: str):
        self.__id = id
        self.__nombre = nombre

    def resumen(self) -> str:
        return f"Usuario: {self.__nombre}, ID: {self.__id}"
```

Usuario
- id: int - nombre: str
+ __init__ (id: int, nombre: str): None + resumen(): str

El signo - indica visibilidad privada y el signo + indica visibilidad pública. El método __init__() inicializa el identificador y el nombre del usuario, mientras que resumen() devuelve una descripción con los datos del usuario.

Ejercicio 4. Diseña la clase Incidente

Define id, descripción, gravedad, estado y responsable. Indica cuál atributo materializa la asociación y por qué responsable puede iniciar en None.

Respuesta / desarrollo:

Código Python:

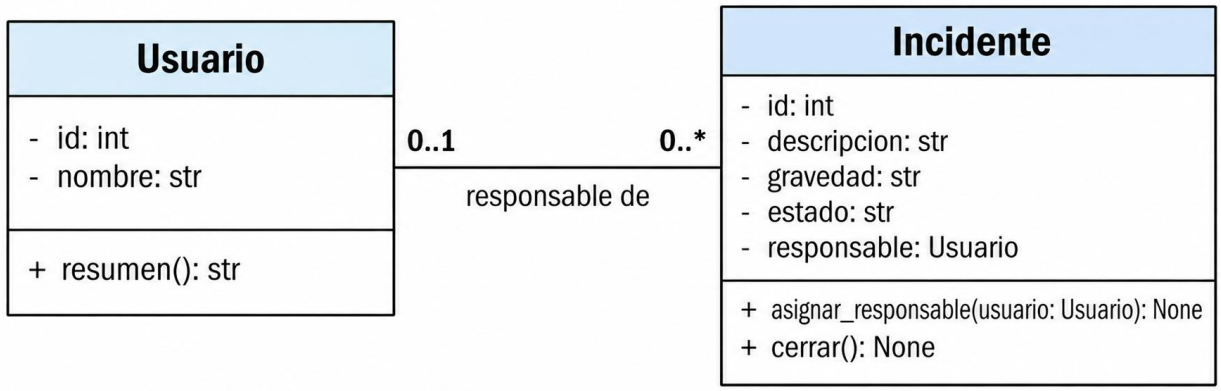
```python
class Incidente:
    def __init__(
        self,
        id: int,
        descripcion: str,
        gravedad: str,
        estado: str = "Abierto",
        responsable: Usuario = None
    ):
        self.__id = id
        self.__descripcion = descripcion
        self.__gravedad = gravedad
        self.__estado = estado
        self.__responsable = responsable
```

Explicación: La clase Incidente contiene los atributos id, descripcion, gravedad, estado y responsable. El atributo responsable materializa la asociación con la clase Usuario, porque almacena una referencia a un objeto de esa clase. Puede iniciar en None porque un incidente puede registrarse sin tener un responsable asignado.

Ejercicio 5. UML de asociación

Dibuja Usuario e Incidente. Agrega una asociación etiquetada “responsable de” y sus multiplicidades. Entrega el diagrama como imagen o PDF.

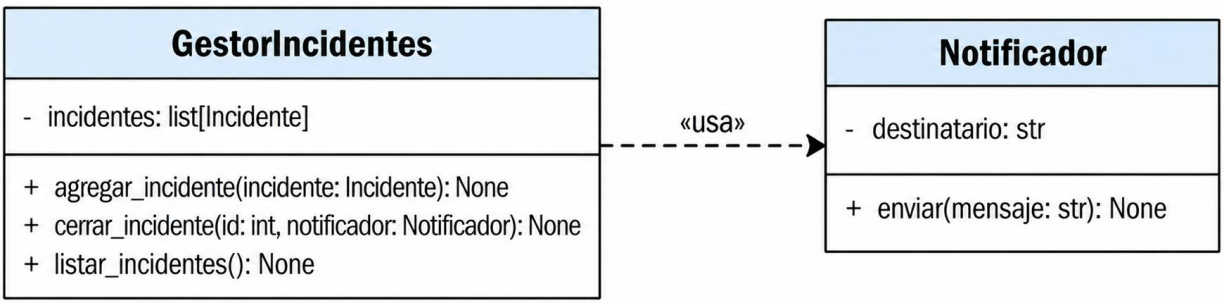
Respuesta / desarrollo:



Ejercicio 6. Dependencia con Notificador

Modela GestorIncidentes y Notificador. GestorIncidentes debe usar Notificador para completar una operación de cierre, sin conservarlo como atributo. Representa la dependencia en UML.

Respuesta / desarrollo:



UML — Dependencia

El diagrama debe mostrar una línea discontinua con flecha abierta desde GestorIncidentes hacia Notificador, etiquetada «usa».

- **- indica atributo privado.**
- **+ indica método público.**
- **La flecha discontinua representa la dependencia.**

Ejercicio 7. Implementación Python

Programa Usuario, Incidente, Notificador y GestorIncidentes. Crea dos usuarios y tres incidentes. Asigna responsables y cierra uno mediante GestorIncidentes.

```python
class Usuario:
    def __init__(self, id: int, nombre: str):
        self.__id = id
        self.__nombre = nombre

    @property
    def id(self):
        return self.__id

    @property
    def nombre(self):
        return self.__nombre

    def resumen(self) -> str:
        return f"Usuario: {self.__nombre}"


class Incidente:
    def __init__(
        self,
        id: int,
        descripcion: str,
        gravedad: str,
        estado: str = "Abierto",
        responsable: Usuario = None
    ):
        self.id = id
        self.descripcion = descripcion
        self.gravedad = gravedad
        self.estado = estado
        self.responsable = responsable
```

```
    def cerrar(self):
        self.estado = "Cerrado"


class Notificador:
    def enviar(self, incidente: Incidente):
        print(
            f"Notificación: {incidente.id} => "
            f"{incidente.estado}"
        )


class GestorIncidentes:
    def cerrar_incidente(
        self,
        incidente: Incidente,
        notificador: Notificador
    ):
        incidente.cerrar()
        notificador.enviar(incidente)


# Crear dos usuarios
usuario1 = Usuario(1, "Daniel")
usuario2 = Usuario(2, "Laura")

# Crear tres incidentes y asignar responsables
incidente1 = Incidente(
    101, "Acceso sospechoso", "Alta", responsable=usuario1
)
incidente2 = Incidente(
    102, "Alerta de seguridad", "Media", responsable=usuario2
)
incidente3 = Incidente(
    103, "Modificacion no autorizada", "Alta",
    responsable=usuario1
)

# Crear gestor y notificador
gestor = GestorIncidentes()
notificador = Notificador()

# Cerrar un incidente mediante el gestor
gestor.cerrar_incidente(incidente1, notificador)
```

```
# Mostrar resultados
print("\n=== USUARIOS ===")
print(usuario1.resumen())
print(usuario2.resumen())

print("\n=== INCIDENTES ===")
for incidente in [incidente1, incidente2, incidente3]:
    print(
        f"ID: {incidente.id} | "
        f"Descripción: {incidente.descripcion} | "
        f"Gravedad: {incidente.gravedad} | "
        f"Estado: {incidente.estado} | "
        f"Responsable: {incidente.responsable.nombre}"
    )
```

Ejercicio 8. Explica el código

En un documento responde: ¿dónde se ve la asociación?, ¿dónde se ve la dependencia?, ¿qué cambiaría si GestorIncidentes almacenara self.notificador?, ¿qué multiplicidad utiliza tu diseño?

1. ¿Dónde se ve la asociación?

La asociación se ve en la clase Incidente, mediante el atributo responsable, que guarda una referencia a un objeto de tipo Usuario. Esto permite asignar un usuario como responsable de un incidente.

2. ¿Dónde se ve la dependencia?

La dependencia se ve en el método cerrar_incidente() de la clase GestorIncidentes, que recibe un objeto Notificador como parámetro y lo utiliza para enviar una notificación cuando se cierra un incidente.

3. ¿Qué cambiaría si GestorIncidentes almacenara self.notificador?

Si GestorIncidentes almacenara self.notificador, conservaría una referencia al objeto Notificador como atributo. La relación sería más permanente y el gestor podría utilizar ese mismo notificador en diferentes operaciones, en lugar de recibirlo como parámetro cada vez.

4. ¿Qué multiplicidad utiliza tu diseño?

La asociación entre Usuario e Incidente utiliza las multiplicidades 0..1 y 0..*. Un incidente puede tener cero o un responsable, mientras que un usuario puede ser responsable de cero, uno o muchos incidentes.